Notes fonctionnelles — BudgetApp

Objectifs

- Suivre les dépenses et revenus
- Catégoriser les transactions
- Générer un résumé du budget

Fonctionnalités à implémenter

1. Créer un user / rôle
2. Ajouter / modifier / supprimer une transaction
3. Catégories personnalisées
4. Graphiques de suivi (revenus vs dépenses)
5. Historique mensuel

Signification des variables CSS Bootstrap

Exemple avec `mb-3` :

- **m** — Définit le type de propriété : Marge (margin).
- **b** — Définit la direction : Bas (bottom).
- **3** — Définit la taille ou le niveau d'espacement.

Le sélecteur de classe `<button class="btn btn-danger w-100">` utilise **trois classes Bootstrap** essentielles :

1. Stylisé comme un bouton standard de Bootstrap (`btn`).
2. De couleur **rouge vif** (`btn-danger`).
3. Étendu sur toute la **largeur** de son conteneur (`w-100`).

> **REMARQUE IMPORTANTE :**

> Le numéro de carte (`fullCardNumber`) doit être chargé dynamiquement depuis le serveur de manière sécurisée. Ne JAMAIS le laisser en clair dans le code HTML ou JavaScript pour la sécurité.

Exemples de données MySQL

```
allTransactions = [
  {
    id: 1,
    date: '2025-09-12',
    description: 'Électricité',
    category: 'Factures',
    source: 'Prélèvement',
    amount: -75.00,
    type: 'debit',
    icon: 'lightning-charge',
    color: 'text-info'
  },
  {
    id: 2,
    date: '2025-09-10',
    description: 'Achat Livres',
    category: 'Dépenses',
    source: 'Paiement carte',
    amount: -22.50,
    type: 'debit',
    icon: 'book',
    color: 'text-danger'
  },
  {
    id: 3,
    date: '2025-09-02',
    description: 'Courses Supermarché',
    category: 'Dépenses',
    source: 'Paiement carte',
    amount: -85.30,
    type: 'debit',
    icon: 'cart',
    color: 'text-warning'
  },
  {
    id: 4,
    date: '2025-09-01',
```

```
    description: 'Salaire Mensuel',
    category: 'Revenus',
    source: 'Virement bancaire',
    amount: 3500.00,
    type: 'credit',
    icon: 'bank2',
    color: 'text-success'
  }
]
```

1. Structure BackEnd du projet

```
/mon-app
├── /assets      (Images, CSS, JS)
├── /includes    (Logique partagée : db.php, fonctions.php)
├── /templates   (Bouts de code HTML : header.php, footer.php)
├── /actions     (Scripts PHP de traitement : login_action.php,
transfer_process.php)
├── index.php    (Point d'entrée / Home)
├── login.php
├── profile.php
└── ...
```

2. Utiliser le système de « Templates »

Au lieu de répéter le code du menu ou du `head` sur chaque page, utiliser des includes.

Exemple dans `home.php` :

```
<?php
require_once 'includes/db.php';       // Connexion BDD
require_once 'includes/auth_check.php'; // Vérifie si l'user est connecté

include 'templates/header.php'; // Contient le <head> et le début du body
?>

    <main class="container">
      <h1>Bienvenue, <?= $_SESSION['user_name'] ?></h1>
    </main>

<?php include 'templates/footer.php'; // Contient les scripts JS et la fin du
body ?>
```

3. Organisation logique par fonctionnalités

4 grands pôles à regrouper dans `actions/` :

Pôle	Pages concernées	Logique Backend à prévoir
1. Authentification	`login.html`, `register.html`	Sessions PHP, hachage de mots de passe, vérification email
2. Transactions	`transfert.html`, `topup.html`, `bankhistory.html`	Requêtes SQL INSERT (transactions), calcul de solde, vérification des fonds
3. Gestion Contacts	`mycontact.html`, `newrecipient.html`, `detailcontact.html`	CRUD sur la table contacts
4. Profil & Cartes	`profile.html`, `cardinfo.html`, `setting.html`	Mise à jour des données user, gestion des états des cartes

4. Passer du .html au .php dynamique

Pour que les liens fonctionnent avec la base de données, changer les extensions dans le menu :

Remplacer `<a href="profile.html">` par `<a href="profile.php">`.

Pour les détails des contacts : ne pas utiliser une page par contact. Utiliser une variable GET.

- Lien : `detailcontact.php?id=45`
- Backend : `$id = $_GET['id'];` — on récupère l'ID pour afficher le bon contact via SQL.

5. Sécurité de base (indispensable)

Puisque l'application gère des transferts et de l'argent :

Sessions : Commencer chaque page protégée par `session_start()`. Si `$_SESSION['user_id']` n'existe pas, rediriger vers `login.php`.

Requêtes préparées : ne jamais insérer de variables directement dans les requêtes SQL.

Pratique	Exemple
Mauvais	`"SELECT * FROM users WHERE id = $id"`
Bon	Utiliser PDO avec des marqueurs `?`

Validation : pour les pages de transfert, toujours vérifier côté serveur que le montant est positif et que l'utilisateur possède bien la somme avant de valider.

Configuration — Section Login via PHP

1. Le fichier de configuration : `includes/db.php`

Ce fichier sera inclus au début de chaque page qui a besoin d'accéder aux données (profil, historique, transferts).

2. Organisation de la Base de Données (SQL)

Table	Colonnes principales
`users`	id, nom, email, password, solde (balance)
`contacts`	id, user_id (le propriétaire), contact_name, account_number, email

Table	Colonnes principales
`transactions`	id, sender_id, receiver_id, amount, type (transfer/topup), date

3. Exemple d'utilisation dans `profile.php`

Maintenant que `db.php` est prêt, utiliser ce fichier pour récupérer les infos de l'utilisateur connecté (voir `profil.php`).

Pourquoi faire comme ça ?

- **Sécurité :** les requêtes préparées (`prepare` + `execute`) empêchent les pirates d'injecter du code dans les formulaires.
- **Maintenance :** en cas de changement du mot de passe de base de données, un seul fichier à modifier (`db.php`).
- **Lisibilité :** le code HTML reste propre avec un code PHP bien rangé.

4. La sécurité : `includes/auth_check.php`

Pour empêcher quelqu'un d'accéder à `profile.php` ou `transfert.php` en tapant simplement l'URL, inclure ce fichier en haut des pages protégées :

```
<?php
session_start();
if (!isset($_SESSION['user_id'])) {
    // Si l'utilisateur n'est pas connecté, retour au login
    header("Location: login.php");
    exit();
}
?>
```

5. Le script de traitement : `actions/register_action.php`

Ce script va vérifier si l'email existe déjà, hacher le mot de passe, et insérer l'utilisateur avec un solde initial de 0 €.

6. Pourquoi c'est important ?

- **`password_hash()` :** même si un pirate accède à la base de données, il ne pourra pas lire les mots de passe. Ils ressembleront à une suite de caractères illisibles comme `$2y$10$vI8...`
- **`filter_var` & `htmlspecialchars` :** ces fonctions protègent l'application contre les attaques XSS (injection de scripts malveillants).
- **`lastInsertId()` :** cette fonction de PDO récupère l'ID généré automatiquement par MySQL, ce qui permet de créer la session immédiatement.

Rappel sur la structure de la table `users`

Colonne	Type / Contrainte
`id`	INT, Auto Increment, Primary Key
`nom`	VARCHAR(100)
`email`	VARCHAR(150), Unique
`password`	VARCHAR(255)
`balance`	DECIMAL(10,2), Default 0.00